Memory Allocation in Multidimensional Dataflow exploiting Ownership Semantics

Pedro Ciambra[1*†], Mickaël Dardaillon[†], Maxime Pelcat[†] and Hervé Yviquel[*]
[*]Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil
Email: pedro.ciambra@ic.unicamp.br, hyviquel@unicamp.br
[†]INSA Rennes, IETR, UMR 6164, Rennes, France
Email: firstname.lastname@insa-rennes.fr

***Abstract*—Dataflow computational models have gained renewed interest due to the rise of data-centric machine learning algorithms. However, these models often struggle to effectively represent information about the storage and exchange of data in memory, especially across multiple dimensions. Such representation is critical for creating optimized schedules and memory mappings, particularly for cache-enabled von Neumann architectures. This paper introduces LADF, an enhanced dataflow model that incorporates multidimensional buffers and ownership semantics as core elements, and bufferflow, a diagram notation for visualizing memory allocation in dataflow. We explore these concepts through a practical example, illustrating their significance in optimizing computational processes.**

***Index Terms*—dataflow, compilation, multidimensional, ownership semantics, bufferflow, ladf.**

I. Introduction

Procedural programming is a familiar paradigm for most developers, characterized by implementing algorithms through a sequence of operations. These operations can read and write to arbitrary regions of a globally accessible memory space and jump to any code region based on the data stored in this memory, aligning closely with the von Neumann architecture prevalent in most general-purpose computers [1]. This paradigm, while flexible and providing reasonably predictable performance, originates from the era of single-core computers. As such, it does not lend itself easily to parallelization. Additionally, procedural programming is challenging to analyze systematically. Arbitrary memory accesses complicate the visibility of crucial factors such as the lifetime of values, memory aliasing, and data dependencies [2]. These elements are critical for the automated generation of accurate and efficient scheduling for parallel algorithms.

Dataflow was created in part to tackle this issue [3]. It reasons about data not by their location in memory, but by the values they carry and the transformations enacted by code. Instead of accessing memory regions, dataflow actors consume and produce data "tokens", which are not associated with any particular memory location at design time. This allows the scheduler to manage the binding of tokens to memory as necessary, enabling performant parallel schedules without special knowledge of the target platform. Dataflow has a wide variety of applications, being useful for high-level synthesis, digital signal processors, distributed systems, and non-conventional computational topologies [4].

However, this high level of abstraction from the memory prevents the foundational dataflow models from being competitive with procedural programming for many applications. In the current landscape of architectures for general-purpose computing, the most successful strategy for the development of high-performance implementations is procedural, parallel cache-aware programming, where knowledge of the target platform can be used to manually structure the memory access of the program in order to minimize cache-related slowdowns.

This requires precise knowledge of memory behavior that cannot be modeled by dataflow, as it has strict separation between topology and kernel implementation. Compiler developers work around this limitation by providing this missing information to the scheduler through other means. Desnos et al. [5] extend the Preesm dataflow compiler [6] with annotations that provide extra information about the behavior of dataflow ports, exposing opportunities for data and memory reuse and allowing kernels to be implemented with algorithms that perform in-place computations (which is the standard for high-performance libraries such as LAPACK [7]).

The ability to expose memory information to dataflow allocators also allows for *slicing*, or the ability to refer to a partial section of a buffer without needing to copy it. While this has been tackled in existing dataflow compilers by the implementation of special behavior for "fork" and "join" actors, they have the limitation of being necessarily memory-contiguous and one-dimensional. Being able to access memory in flexible layouts is essential for data-oriented multidimensional algorithms such as convolutions, which execute the same computation on many sub-regions of few buffers, where data locality matters and copies can be very costly. This phenomenon has led dataflow projects that rely on multidimensional algorithms (such as most modern machine learning frameworks) to delegate the memory-critical access to procedural programming languages, giving up the advantages of a more coarse-grained automated scheduler.

This context leads us to the conclusion that there is a need for a new, lower-level dataflow model that accounts for memory behavior and multidimensional data layout. While nontrivial, these features are not so complex as to completely negate the advantages of dataflow programming, and could enable it for a new class of algorithm.

A. Multidimensional data

Classical dataflow models of computation represent the transfer of data between actors with FIFO semantics; nodes consume tokens from the fronts of input queues and append to the backs of output queues. While this achieves some flexibility when generating a schedule and enables implicit data parallelism, this has the limitation of being strictly one-dimensional and only provides a single way to distribute data between tasks. Any special addressing would require a new actor that produces the data in the expected shape, which adds otherwise unnecessary copies and scheduler pressure.

Conversely, in multi-dimensional algorithms, it is often necessary to split buffers into arbitrarily sized components and along any of the data dimensions, which may result in complex memory access layouts. These partial references can be copied or modified and eventually recombined into the original buffer. A buffer may also have many overlapping references being addressed simultaneously, and the lifetime of its value must be managed to ensure correct data dependency.

While multidimensional references are something that has become common in modern general-purpose programming languages, not many dataflow works explore this type of representation. A notable example is ArrayOL [8], which extends dataflow by allowing the representation of multidimensional data access using linear algebra and the concept of repetition for describing data parallelism.

B. Ownership and borrowing

Memory management continues to be a difficult problem in both industry and academia. One of the main goals being pursued is automated memory reuse: how to detect that a particular value will not be referenced again. This allows the corresponding memory to be reclaimed and reused by whichever memory management solution is being employed by the program. This problem becomes more complex when memory aliasing is introduced; when we allow memory references such as pointers to be transformed, copied and mutated, we must keep track of all other references to the same memory region. We can do this either manually, relying on conventions and documentation to register which section of code is responsible for freeing the memory once used (as in C); delegate it to a "garbage collection" algorithm that will, at runtime, periodically run through the entire memory to find unreferenced values (as in Java or C#); or, more recently, rely on high-level ownership semantic features that have been made available in languages such as Rust [9]. This last strategy has the advantage of lacking both the overhead of a garbage collection algorithm and the requirement of manual programmer intervention, while having the downside of a somewhat constrained programming style that discourages the heavy use of indirection. Nevertheless, the fact that it prevents faulty manual memory management while retaining performance has greatly influenced the rise in popularity of ownership semantics.

One of the hallmarks of an ownership system is type system-based lifetime tracking. Where traditional programming languages allow opaque pointers that can be used regardless of the dependencies of the pointed-to data, a language with ownership semantics tracks variables in terms of the amount, type, and validity of references into its corresponding memory. A variable can be "owned", which means its lifetime is that of the scope it finds itself in and it will be freed once the scope ends; it can be a read-write reference, which does not have responsibility over the memory and will not cause it to be freed; or it can be a read-only reference, which adds the requirement that the function is not allowed to write into the memory in question. To guarantee the correctness of the memory access, the go-to strategy is to enforce *borrow semantics*: an owned variable may be temporarily converted into either a single read-write reference, or one or more read-only references, but not both kinds at the same time. While this adds an extra constraint for the programmer that complicates the implementation of heavily pointer-relying algorithms such as sparse data structures, it simplifies the problem of aliasing to a level that is manageable by an automatic system.

Having the type system track this extra information is also useful from an optimizing compiler's point of view, as it allows optimizations the freedom to insert and remove copies and to move data in memory, while preserving correctness.

Mapping these semantics to the perspective of dataflow models of computation, each token acts as an owned variable that is consumed, and thus its memory freed, as soon as the consuming actor finishes its execution; annotations such as Preesm's memory script allow using read-write references instead of owned variables for in-place actors, and read-only references for broadcasts.

In this work, we propose Layout Aware Dataflow (LADF), a static dataflow model that tackles the issue of multidimensional memory references by using ownership and borrow semantics to manage data allocations and deallocations, allowing for slicing in arbitrary dimensions and avoiding unnecessary copies and memory usage. To illustrate that, we introduce *bufferflow* diagrams that augment dataflow graphs with information on the memory allocation choices performed by the scheduler.

II. Memory management in dataflow

Schedulers for classical dataflow models rely on the assumption that data is written and read only once; a straightforward allocation schedule consists of allocating the memory immediately before the production of the value and frees it immediately after its consumption, while special actors like *fork*, *join* and *broadcast* may be optimized for multiple reads and linear splitting of values. Depending on the specific dataflow model of computation in question, this memory mapping may occur at compile-time or at runtime. This work will focus on runtime memory allocation, but many of the arguments also apply to static allocation schemes based on symbolic execution.

For the purposes of illustration, consider the dataflow graph in Fig. 1. Here, actor Read emits a single token that contains an RGB color point. The Broadcast actor emits three copies of this input token. H, L and S each read one copy and emit the corresponding component of a HSL color point.

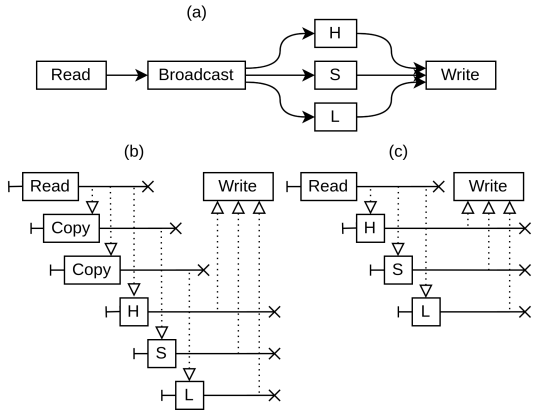

Fig. 1: (a) Dataflow graph with broadcast node. (b) Bufferflow diagram showing memory allocation where the original token is reused once. (c) Bufferflow diagram showing memory allocation with borrow semantics, no copies necessary.

In traditional dataflow scheduling, the broadcast node generates at least two copies, as tokens cannot be divorced from their corresponding memory, and are freed after the execution of the consumer.

To visualize the behavior of this dataflow schedule in regards to memory, we propose the diagram in Fig. 1b, where horizontal lines correspond to a specific buffer in memory, and the left and right end-stops represent their allocation and deallocation in time, respectively. Actors overlapping a buffer alter the contents of that buffer. Vertical dotted lines represent read-only access of a memory region. Here, we see that the broadcast actor generated two allocations. Similarly, H, S and L each generate an allocation in which to store their outputs. Finally, after going through the Write actor, each allocated region is freed. This form of diagram, henceforth referred to as "bufferflow", retains the dependency semantics of dataflow, while adding information about memory allocation and access.

By adding the constraint that actors cannot change the memory assigned to their read-only input ports, we avoid the copies and allocations generated by adding a shared lifetime to the tokens, and by having each node read from the original memory location (Fig. 1c). This means that, rather than simply scheduling the freeing of the memory of each input port after the execution of the actor, we schedule the free only after all reading actors have executed. Since actors may be executing concurrently, this requires synchronization, which, depending on the scheduling and allocation scheme being used, may incur its own overhead. Therefore, we have a trade-off between the overhead of copies plus extra memory consumption and the overhead of synchronization.

While the constraint of read-only input ports is not a big departure from classical dataflow, the inclusion of read-write ports is. While the implementations for classical dataflow actors can be provided in the form of functions where each port maps to a non-overlapping memory region, read-write actors contain one or more pairs of input and output ports that refer to the same underlying memory. This allows the scheduler to prevent unnecessary copies, as long as dependencies are respected.

When a broadcast node flows into multiple read-write ports,

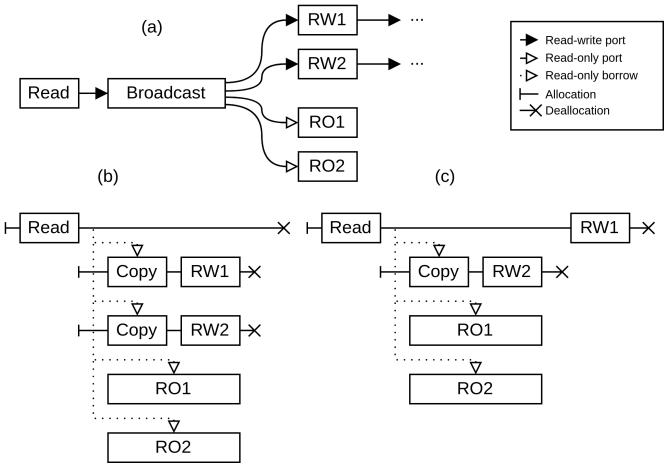

Fig. 2: (a) A broadcast node with two read-only ports and two read-write ports. (b) Scheduling copies for both read-write ports. They can be executed in parallel with the other consuming actors. (c) Reusing original buffer for read-write port RW1. It must execute after the read-only borrows finish.

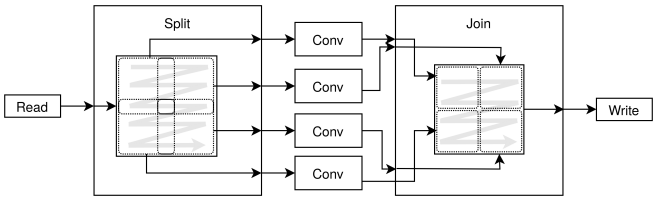

Fig. 3: Parallelization of a convolution. Four overlapping sections of the data must be processed by different instances of the Conv actor, and the results must be joined together into a contiguous image. Memory contiguity is represented by the gray arrows.

extra copies must be generated, transforming read-write ports into read-only ports; the choice of node to which assign the original token is an additional design space decision to be solved by the scheduler. If a broadcast node flows into read-only ports, there is a choice of generating copies, or adding extra logical dependencies to ensure that the read-write actor executes strictly after the read-only actors (Fig. 2).

III. Multidimensional Dataflow

Because traditional dataflow models of computation represent the connections between actors as data streams, the representation of multidimensionality is typically delegated to the implementation of the actors. In the case of 2D data, this consists of a row-major contiguous representation, sometimes parameterized on the dimensions of the matrix. While effective, this hides potentially useful layout information from the dataflow compiler, particularly when it comes to exposing parallelism. Consider the parallelization of a 2D convolution in Fig. 3.

Even though convolutions are highly parallelizable operations, the fact that data dependencies overlap presents a challenge to an automated scheduler. Splitting the input image into multiple parts requires aliasing their overlap, which brings us back to the issue of ownership; the buffer may only be deallocated after all readers have finished executing. Because a traditional dataflow scheduler only has local information about buffer ownership and cannot reason about data dependency

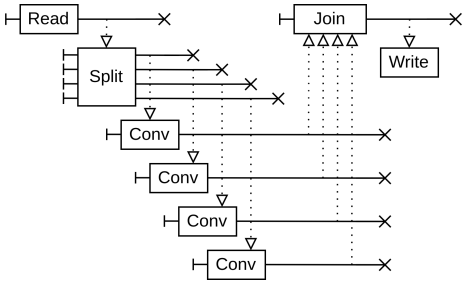

Fig. 4: Memory allocation for Fig. 3 in traditional Dataflow. Each horizontal line corresponds to a separate memory allocation. The Split actor copies each quadrant into a separate contiguous buffer, and Join copies the results of Conv into a new buffer. These actors must be implemented manually.

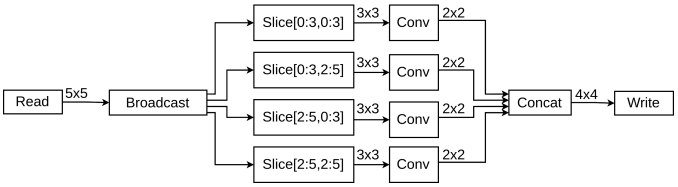

Fig. 5: Convolution of a 5x5 matrix with with Slice actors and a Concat actor in LADF. Slices represented in NumPy notation.

with non-contiguous memory access, as is the case for multi-dimensional slices, this can only be done manually by copying the sections into a contiguous intermediary buffer (in which case the lack of ownership semantics may lead to numerous allocations and copies, as shown in Fig. 4) or with a more fine-grained technique such as polyhedral analysis, which can drastically raise compile times.

With this in mind, we define LADF by introducing into the dataflow model the concept of multidimensional slices: a multi-coordinate view into a buffer that maps into a tensor of arbitrary shape, by way of coordinate transformations. This type of reference is common in numerical processing contexts, being employed by programming languages such as MatLab, R, and Julia, and in frameworks such as NumPy. In the context of dataflow, this has been approached in works such as [8] and [10], but not in the context of memory allocation.

By allowing the type system to carry layout information, we may rewrite the algorithm as Fig. 5. The data layout of the token emitted by the Read actor is encoded as a multidimensional shape; actors that read this token have static access to this layout and can correctly process its contents. Slice actors represent a coordinate transformation and do not generate copies, propagating the read-only reference to their output; similarly, the Concat actor tells the scheduler to allocate the memory in the final configuration and write into it directly, avoiding copies. They are multidimensional generalizations of the Fork and Join dataflow actors, respectively.

This graph generates the schedule in Fig. 6, which performs no extra allocations other than the input and output buffers. The four 3x3 slices, whose extents are represented in NumPy notation, are aliases for the memory of the input buffer. As all of the consuming ports are read-only, the slices may overlap, according to the borrow rules. The dotted lines with solid arrows represent *mutable* borrows, where control over the contents of that section of the output buffer is given to the

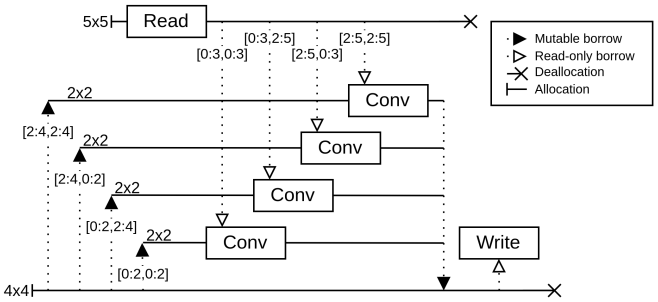

Fig. 6: Memory allocation for the parallelized convolution with slicing and borrowing. Slices represented in NumPy notation.

Conv actor. This is allowed because the four 2x2 slices do not overlap. After the Conv actors execute, write access is "returned" to the buffer, before it is finally read by the Write actor.

IV. Conclusion

In this work we introduce bufferflow, a graphical representation for memory allocation in dataflow graphs. We use it to represent memory allocation in LADF, a new model of computation that raises multidimensional memory information into the dataflow design space. By harnessing ownership semantics, we enable flexible memory optimizations in dataflow that avoid extraneous copies and memory usage when tiling multidimensional algorithms. Future work will explore the application of LADF to automatic tiling in a dataflow context.

References

[1] J. von Neumann, "First draft of a report on the edvac," tech. rep., University of Pennsylvania, 1945. Originally published as a manuscript.

[2] A. Aho, M. Lam, R. Sethi, and J. D. Ullman, "Compilers: Principles, techniques and tools, 2nd edition," 2007.

[3] E. A. Lee, "The problem with threads," Computer, vol. 39, pp. 33–42, 2006.

[4] V. K. Madisetti and I. T. Young, The Digital Signal Processing Handbook-3 Volume Set. CRC press, 2018.

[5] K. Desnos, M. Pelcat, J. F. Nezan, and S. Aridhi, "On Memory Reuse Between Inputs and Outputs of Dataflow Actors," ACM Transactions on Embedded Computing Systems (TECS), vol. 15, p. 30, Feb. 2016. Publisher: ACM.

[6] M. Pelcat, K. Desnos, J. Heulot, C. Guy, J.-F. Nezan, and S. Aridhi, "Preesm: A dataflow-based rapid prototyping framework for simplifying multicore dsp programming," in 2014 6th european embedded design in education and research conference (EDERC), pp. 36–40, IEEE, 2014.

[7] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, LAPACK Users' Guide. Philadelphia, PA: Society for Industrial and Applied Mathematics, third ed., 1999.

[8] P. Boulet, "Array-OL Revisited, Multidimensional Intensive Signal Processing Specification," tech. rep., 2007.

[9] N. D. Matsakis and F. S. Klock II, "The rust language," in ACM SIGAda Ada Letters, vol. 34, pp. 103–104, ACM, 2014.

[10] P. de Oliveira Castro, S. Louise, and D. Barthou, "A multidimensional array slicing dsl for stream programming," 2010 International Conference on Complex, Intelligent and Software Intensive Systems, pp. 913–918, 2010.